

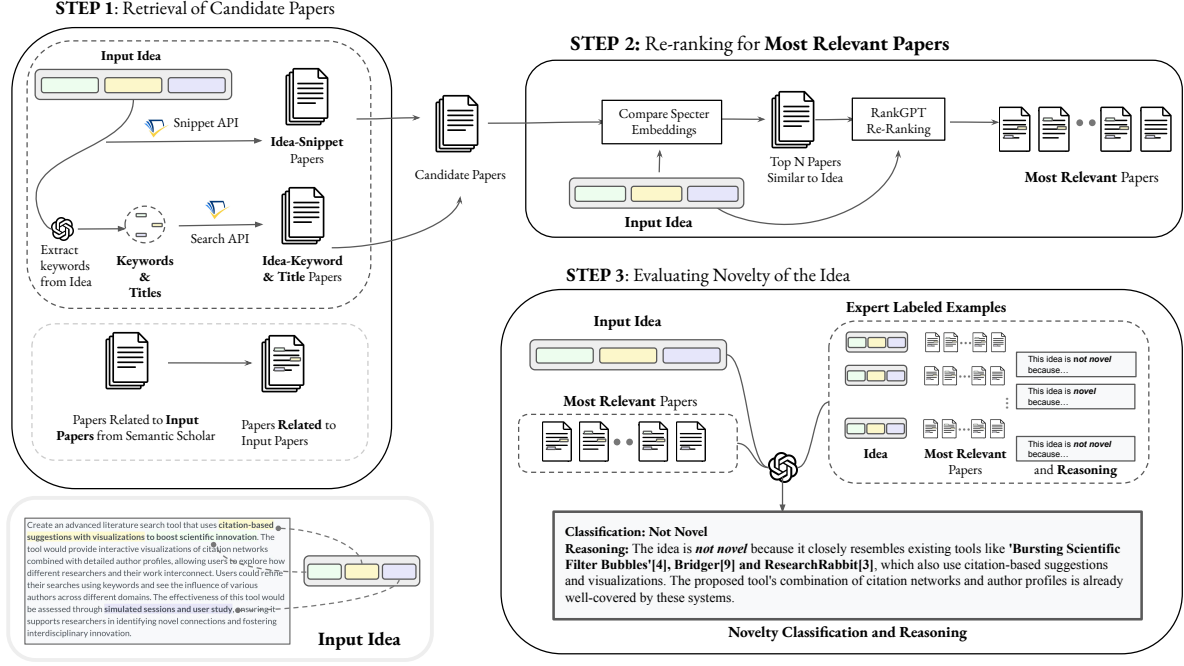


Figure 1: Our Idea Novelty Checker follows a retrieve-then-rerank approach for novelty evaluation. First, it gathers a broad set of papers relevant to an idea using query expansion (extracting keywords and titles from the idea) and snippet search (using the entire idea as input). Optionally, if seed papers are provided, we retrieve papers similar to these seed papers. Next, a two-stage re-ranking process is applied, where an embedding-based ranking strategy filters the large collection to top- N papers, followed by a facet-based LLM re-ranker to identify the top- k most relevant papers. Finally, these top- k papers are used to assess the idea’s novelty, guided by in-context examples that evaluate novelty with grounded reasoning.

Lamothe; Meng et al.; Sun et al.; Baldelli et al.), our pipeline uses a two-phase approach for identifying the most relevant papers to a given idea. First, we gather a broad set of candidate papers related to the idea. Then, we re-rank these candidates in two steps: first using embedding-based similarity, and then applying LLM-based re-ranking to facet-based similarity.

STEP 1: Retrieval of Candidate Papers

To accurately assess the novelty of an idea, it is crucial to compare it against a comprehensive collection of papers that cover the various facets of the idea. For a given idea and its corresponding papers (if any) used to generate the idea, we find more related papers to these input seed papers using the Semantic Scholar API³. However, simple retrieval methods often overlook important aspects of an idea (Mysore et al., b,a; Wang et al., a). To improve the paper collection’s coverage we follow (Lu et al.; Si et al.) and employ a query-based retrieval method, where search queries are generated corresponding to different keywords related to the

idea, and queried through the Semantic Scholar Search API (Kinney et al.). Corresponding to each search query, papers are added to the collection of relevant papers. We prompt the LLM (LLM_{query}) to generate these search queries based on the keywords and potential titles related to the idea.

Next, we also employ Semantic Scholar’s snippet search⁴, which is trained to identify similar snippets (approximately 500 words of text) in other papers. We leverage the context size of this retrieval mechanism by incorporating the entire idea into the snippet search. Finally, we combine the seed papers and their related works with the papers retrieved from the two Semantic Scholar based query-retrieval method. This combined set form the candidate papers for the ideation process.

STEP 2: Re-ranking for Most Relevant Papers

To identify the papers most likely to overlap with the candidate idea, we implement a two-stage re-ranking process that combines embedding-based filtering with an LLM-based re-ranking approach. To identify the papers most likely to overlap with

³api.semanticscholar.org/api-docs/recommendations

⁴api.semanticscholar.org/api-docs/#tag/Snippet-Text

the candidate idea, we implement a two-stage re-ranking process that combines embedding-based filtering with an LLM-based re-ranking approach.

First, we employ **embedding-based filtering** to compute the semantic similarity between the idea and each paper in our collection of papers from STEP 1. We select the top N papers with the highest cosine similarity between their embeddings and the idea embedding. While this embedding-based ranking efficiently narrows down the collection of papers, it is limited in its capacity to capture deeper and more contextual relationships between different facets of the idea and the papers, in comparison to powerful state-of-art LLMs (Reimers and Gurevych).

To address these limitations we employ a popularly used **LLM-based re-ranker**, RankGPT (Sun et al.), which refines the initial ranking of candidate papers by examining how relevant each paper is to the idea. We change relevance criteria to match it with each key facet of the idea. RankGPT goes beyond simple surface similarities by comparing the papers against the idea’s application domain, purpose, mechanism, and evaluation. It follows a clear set of priorities: first, it favors papers that match all key facets of the idea; then, it prefers those that align with the application domain and purpose; next, it considers papers that share similarities in purpose, mechanism, or evaluation; and finally, it ranks lower those that only partially match or address related facets. This approach ensures that the final ranking accurately reflects the relevance and depth of each paper in connection with the idea. We refer to the LLM used for RankGPT as ($LLM_{rankgpt}$).

This collection of k -most relevant papers is used by the novelty checker in the next step to evaluate the idea’s novelty.

4.2 Idea Novelty Evaluation

To assess an idea’s novelty, we prompt an LLM ($LLM_{novelty}$) with both the idea and its top- k relevant papers. The LLM outputs a binary classification (novel or not novel) accompanied by reasoning based on the top- k retrieved literature. To guide the LLM’s judgment, we include $n_{examples}$ in-context examples drawn from our formative study, where $n_{examples}$ is treated as a hyperparameter. These examples reflect the experts’ criteria for novelty: an idea is considered novel if it differs from all retrieved papers in at least one core facet—namely, purpose (i.e., objective of idea), mechanism (i.e.,

technical approach), evaluation (i.e., validation method), a unique combination of these facets, or if it applies the same facets to a new application domain.

5 Implementation & Baselines

Dataset: From our formative study, we collected 67 consensus-labeled examples (39 labeled as novel and 28 as non-novel). We split into training and test sets (35 for training and 32 for testing) with a balanced distribution of novel and non-novel ideas. Please refer to Table 4 in the Appendix for sample examples.

Baselines: We evaluated multiple baselines to benchmark our novelty assessment approach. First, we employed a zero-shot prompt as a straightforward baseline, and further refined this manually written prompt using Anthropic’s prompt generator⁵. We also applied popular prompt optimization techniques such as DSPy (Khatab et al.) and TextGRAD (Yuksekgonul et al.), which optimize the prompt instructions using a train/validation split created from formative study examples.

As an alternative to using in-context examples from the formative study, we extracted reviews from ICLR and NeurIPS submissions via the **Open-Review** API. These reviews comprise aspects such as *strengths*, *presentation*, *limitations*, *soundness*, *weaknesses*, *questions*, *confidence*, *contribution*, *summary*, and *rating*. The input title and abstract were adapted to match the ideas in the training data using a style-change prompt⁶. After rigorous filtering, we identified approximately 8,156 submissions discussing idea novelty and manually selected reviews that specifically evaluated the core idea rather than the entire paper. From these, we randomly sampled 20 idea-review pairs to serve as an additional baseline with different in-context examples.

In addition to these baselines, we also compare our novelty checker ‘prompt’ with that of AI Scientist (Lu et al.) (different from its paper reviewer) and AI Researcher (Si et al.) on the same test set of ideas and fixed top 10 papers. We compare **only the prompts to assess novelty** of these two approaches with ours, rather than the entire system, because the test set containing the novelty judg-

⁵<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-generator>

⁶All prompts are provided in the anonymised codebase.

ments by experts were based on a fixed set of the 10 most relevant papers for each idea. Since different retrieval methods could introduce new papers and potentially change novelty classification, we standardize the most relevant papers to ensure a fair comparison of the prompts alone. Additionally, since both setups require a different style of input idea, we adapted the ideas to match the requirements of each system.

Implementation Settings: For our novelty evaluation system, we use SPECTER-2 (Cohan et al.) as the default embedding model. Initially, we retrieve the top $N = 100$ papers using these embeddings, from which the top $k = 10$ most relevant papers are selected for comparison with the input idea. The default language model for the idea keyword extraction (LLM_{query}), re-ranking process ($LLM_{rankgpt}$), and novelty evaluation ($LLM_{novelty}$) is gpt-4o⁷. Expert-labeled data from the formative study is incorporated as in-context examples in the novelty checker. We experimented with various numbers of in-context examples (comprising idea-paper pairs along with their novelty class and reviews) and found that the best performance was achieved using 15 idea examples (random seed 100). For the OpenReview examples, the best setup involved 5 idea-review pairs. For DSPy we used 2 bootstrapped examples, and trained both DSPy and TextGRAD for 12 prompt iterations.

6 Experiments

In this section, we first compare different baselines on the dataset for novelty evaluations (Section 6.1). Next we present our findings from ablation studies that shows the importance of each component in our approach (Section 6.2). ablations studies We supplement these findings with qualitative examples of expert-labeled ideas and compare our setup with recent novelty checkers (Section 6.3). We conclude with insights from prompt optimization experiments that highlights the sensitivity of LLMs to prompt variations for novelty evaluation tasks (Section 6.4).

6.1 Comparing Novelty Checker Prompts

Our experiments show that incorporating expert-annotated data as in-context examples significantly enhances novelty classification accuracy compared

to zero-shot prompts, DSPY, TextGRAD, and setups using OpenReview examples (Table 1). Since OpenReview reviews do not reference the associated papers, we evaluated our expert-labeled examples both with and without including relevant papers to ensure a fair comparison. Notably, even when we excluded the relevant papers from the expert-labeled examples, our approach still outperformed the OpenReview baseline.

Additionally, we compared two configurations for DSPY, one with reasoning and one without. Our expert-labeled prompt consistently achieved higher performance than the prompt optimizations produced by these methods, and we posit that the number of examples for train/validation were not sufficient for prompt optimisers with gpt-4o. The TextGRAD prompt optimiser did not improve upon its initial system prompt. It provided valuable insights into the LLM’s prompt sensitivity, which we further discuss in Section 6.4.

Our approach achieved over **10 times more agreement with expert-labeled examples** compared to AI Scientist, and **approximately 13% higher agreement** than AI Researcher, further validating the effectiveness of our novelty checker. It is important to note that AI Scientist defaults to "not novel" when it fails to reach a conclusion in novelty evaluation (18 out of 32 times), which may have impacted its agreement rates. We also present some qualitative examples in Figures 2, 3 and 4 of the Appendix, showcasing how these approaches evaluate the novelty of an idea.

6.2 Ablation Studies

Setup: To assess the contribution of each component in our novelty checker, we conducted ablation studies using 58 ideas (comprising 13 ‘not novel’ instances from our test set and 45 NLP papers from the literature). For this experiment, we focus on the ‘not novel’ cases, since the ideas labeled novel in expert-labeled test data can vary with different retrieved paper sets. In our ablations, we considered the following variations: (i) **Complete System:** Uses both keyword and snippet retrieval (each returning the top- k documents based on Semantic Scholar’s ranking), embedding filtering, and facet-based RankGPT re-ranking; (ii) **RankGPT Relevance:** Used the same retrieval methods (keyword and snippet) plus embedding filtering, but replaced the facet-based RankGPT re-ranker with one based on general relevance (Sun et al.). This variation differs from the complete system only in the LLM

⁷We used the model "gpt-4o" during August and September 2024.